
THUMALIEN

Pipeline NLP de detection de fake news sur Bluesky

01 Cahier Des Charges Techniques

Formation : Mastere Big Data & IA - Sup de Vinci

Equipe : Azelie Bernard, Sebastien Lazcanotegui

Annee : 2025-2026

Version : Mai 2026

Cahier des Charges Techniques

Projet Thumalien - Social Media Intelligence & AI Monitor

Reference : CDC-THUM-2026-001

Version : 2.0

Statut : Valide

Date de creation : Avril 2026

Chef de Projet : Direction Projet Data

Auteur du systeme : Azelie Bernard - Master Big Data

1. Objet du document

Le present cahier des charges techniques (CDC) formalise les exigences fonctionnelles, techniques et operationnelles du projet Thumalien. Il constitue le document de reference pour l'ensemble des intervenants du projet et sert de base contractuelle pour la validation des livrables.

- Ce CDC couvre :
- Les exigences fonctionnelles et non-fonctionnelles
 - L'architecture technique cible
 - Les contraintes de performance et de securite
 - Les criteres d'acceptation de chaque composant
 - Les interfaces entre les modules

2. Contexte et enjeux

2.1 Contexte general

La proliferation de la desinformation sur les reseaux sociaux constitue un enjeu majeur pour les democraties, les organisations et les citoyens. Les plateformes decentralisees comme Bluesky (protocole AT) offrent un terrain d'observation privilegie grace a leur API ouverte, mais posent de nouveaux defis en termes de volume, de vitesse et de multilinguisme.

2.2 Enjeux strategiques

Enjeu	Description	Priorite
Detection precoce	Identifier les contenus potentiellement trompeurs avant leur viralite	Critique

Analyse emotionnelle	Comprendre l'ambiance des discussions pour detecter les signaux faibles	Haute
Multilinguisme	Supporter au minimum le francais et l'anglais	Haute
Transparence	Chaque prediction doit etre explicable et auditable	Haute
Frugalite	Minimiser l'empreinte carbone des traitements IA	Moyenne
Conformite	Respecter le RGPD et anticiper l'AI Act europeen	Critique

2.3 Perimetre du projet

Inclus dans le perimetre :

- Collecte automatisee des posts publics Bluesky
- Pipeline NLP de detection de fake news (classification binaire : fiable/suspect)
- Modele d'analyse emotionnelle (7 classes)
- Dashboard interactif de visualisation et d'analyse
- Infrastructure conteneurisee (Docker)
- Monitoring de l'empreinte carbone

Exclus du perimetre :

- Verification factuelle du contenu (fact-checking humain)
- Moderation automatisee (suppression de contenus)
- Collecte sur d'autres plateformes (X, Facebook, TikTok)
- Profilage des utilisateurs
- Decisions automatisees a caractere juridique ou medical

3. Exigences fonctionnelles

3.1 Module de collecte (COL)

ID	Exigence	Priorite	Critere d'acceptation
COL-01	Le systeme doit collecter les posts publics Bluesky via l'API AT Protocol	Critique	Connexion etablie, posts recus et stockes
COL-02	La collecte doit couvrir au minimum 28 termes de recherche en francais et 16 en anglais	Haute	44 termes configures et actifs

COL-03	Le systeme doit supporter une collecte continue (24/7) avec redemarrage automatique	Haute	Uptime > 95% sur 30 jours
COL-04	La deduplication doit empecher le stockage de posts identiques	Haute	0 doublons sur un echantillon de 10 000 posts
COL-05	Le collecteur doit gerer les erreurs reseau avec backoff exponentiel (3 tentatives minimum)	Haute	Reprise automatique apres coupure de 5 min
COL-06	Chaque post doit etre enrichi de metadonnees : URI, CID, auteur, date, terme de recherche, date de collecte	Critique	100% des champs presents
COL-07	Le collecteur doit respecter les rate limits de l'API Bluesky	Critique	Aucun ban sur 30 jours
COL-08	Les metriques de collecte doivent etre accessibles (volume/heure, erreurs, latence)	Moyenne	Logs structures disponibles

3.2 Module de stockage (STO)

ID	Exigence	Priorite	Critere d'acceptation
STO-01	Les donnees doivent etre stockees dans MongoDB (base NoSQL documentaire)	Critique	Base operationnelle, CRUD fonctionnel
STO-02	La base doit supporter > 500 000 documents sans degradation de performance	Haute	Temps de requete < 1s pour les aggregations principales
STO-03	Les index doivent couvrir les requetes principales : temporelles, par terme, par langue	Haute	Explain plan confirme l'utilisation des index
STO-04	Un mecanisme de suppression sur demande doit etre disponible (droit a l'effacement RGPD)	Critique	Suppression par URI ou author_handle en < 5s
STO-05	La persistance des donnees doit etre assuree via volume Docker	Haute	Donnees intactes apres redemarrage du conteneur

STO-06	Les enrichissements IA (score, emotion, label) doivent être stockés avec le post original	Haute	Champs ai_score_credibility, ai_emotion, prediction_label présents
--------	---	-------	--

3.3 Module de detection de fake news (DET)

ID	Exigence	Priorite	Critere d'acceptation
DET-01	Le pipeline doit classer les textes en FIABLE (0) ou SUSPECT (1)	Critique	Prediction binaire fonctionnelle
DET-02	Le score de credibilite doit être continu entre 0 et 1	Critique	Probabilite calibree (ECE < 0.10)
DET-03	Le F1-score sur le jeu de test holdout doit être >= 0.85	Critique	Valide par cross-validation 5-fold
DET-04	Le pipeline doit supporter les textes en français ET en anglais	Haute	F1 FR >= 0.80, F1 EN >= 0.85
DET-05	Le pipeline doit supporter les textes courts (< 30 mots, type posts reseaux sociaux)	Haute	F1 sur textes courts >= 0.75
DET-06	La vectorisation TF-IDF doit utiliser 30 000 features maximum avec n-grams (1,3)	Haute	Parametres confirmes dans le modele sauvegarde
DET-07	Les 15 features linguistiques (12 originales + 3 V4/V5) doivent être calculees pour chaque texte	Haute	Extraction fonctionnelle, pas de NaN
DET-08	Le seuil de decision doit être configurable (default : 0.44)	Haute	Parametre modifiable sans retraining
DET-09	Le temps d'inference doit être < 100ms par texte	Moyenne	Benchmark sur Apple M4 Pro
DET-10	Chaque prediction doit pouvoir être expliquee (top mots, features linguistiques)	Haute	Fonction explain_prediction() fonctionnelle

DET-11	Le dataset d'entrainement V5 doit contenir $\geq 100\,000$ textes (articles + textes sociaux + augmentation FR courte + posts FR sociaux synthetiques)	Haute	197 782 textes verifiés
DET-12	Le biais Reuters doit être éliminé du dataset d'entrainement	Critique	Audit confirmant 0% de marqueur Reuters résiduel

3.4 Module d'analyse emotionnelle (EMO)

ID	Exigence	Priorite	Critere d'acceptation
EMO-01	Le modele doit classifier les textes selon 7 emotions : colere, degout, joie, neutre, peur, surprise, tristesse	Critique	7 probabilites produites pour chaque texte
EMO-02	Le modele doit être un MLP PyTorch avec embeddings appris	Haute	Architecture conforme (Embedding -> FC -> Softmax)
EMO-03	Le F1 macro doit être ≥ 0.60 sur le jeu de test	Haute	Valide sur benchmark d'evaluation
EMO-04	Le modele doit supporter les textes en français et en anglais	Haute	Vocabulaire bilingue (25 000 tokens)
EMO-05	Les probabilites emotionnelles doivent être utilisables comme features dans le pipeline de detection	Haute	Integration fonctionnelle (7 features supplementaires)

3.5 Module Dashboard (DASH)

ID	Exigence	Priorite	Critere d'acceptation
DASH-01	Le dashboard doit être accessible via navigateur web (Streamlit)	Critique	URL http://localhost:8501 fonctionnelle
DASH-02	Le dashboard doit proposer 3 pages : Vue Globale, Analyse Temps Reel, Metriques & Transparence	Haute	Navigation fonctionnelle entre les 3 pages
DASH-03	La page Vue Globale doit afficher les KPI principaux : nombre de posts, % fiable, score moyen, repartition linguistique	Haute	4 KPI visibles et à jour

DASH-04	La page Analyse Temps Reel doit permettre l'analyse d'un texte libre avec affichage du score, du verdict et de l'emotion dominante	Critique	Prediction en < 3s apres soumission
DASH-05	Le dashboard doit fonctionner en mode demo si MongoDB est indisponible	Haute	15 posts de demonstration charges
DASH-06	L'explicabilite doit etre affichee : top mots, features linguistiques, mots sensationnalistes detectes	Haute	Section explicabilite visible pour chaque analyse
DASH-06b	La decomposition exacte du meta-learner V8 (?·x) doit etre affichee pour chaque prediction	Haute	Bar plot Plotly + top 3 contributeurs avec direction SUSPECT/FIABLE
XAI-01	Le pipeline doit produire SHAP global (beeswarm + dependence) sur V6	Haute	Figures dans docs/figures/xai/
XAI-02	La fidelite des explications (faithfulness) doit etre quantifiee : AOPC, Comprehensiveness@k, Sufficiency@k vs baseline aleatoire	Haute	AOPC uplift > +0.10 sur le gold set
XAI-03	L'attribution causale au niveau token doit etre disponible sur CamemBERT (Layer Integrated Gradients via Captum)	Moyenne	Heatmap par token + ?_convergence < 0.15 (niveau indicatif minimum)
XAI-04	L'attention CamemBERT doit etre visualisable (CLS dernière couche + heatmap par couche) sur TP, FP, FN	Moyenne	1 figure par classe d'erreur, pour analyse qualitative
XAI-05	Les explications doivent etre validees par audience : utilisateur, modérateur, régulateur, chercheur	Haute	Model card docs/12_model_card.md section 7.4
DASH-07	Le theme doit etre dark avec accents cyan et effet glassmorphism	Moyenne	Conformite visuelle validee
DASH-08	Les graphiques doivent etre interactifs (zoom, hover, export) via Plotly	Moyenne	Interactions fonctionnelles
DASH-09	Le contraste texte/fond doit respecter WCAG 2.1 AA (ratio >= 4.5:1)	Haute	Valide par outil d'audit accessibilite

4. Exigences non-fonctionnelles

4.1 Performance

ID	Exigence	Critere
PERF-01	Temps d'entrainement du pipeline complet	< 10 minutes sur Apple M4 Pro
PERF-02	Temps d'inference par texte	< 100ms (unitaire), < 5s pour 1 000 textes (batch)
PERF-03	Temps de chargement du dashboard	< 5s (premiere page)
PERF-04	Temps de reponse des requetes MongoDB (aggregations dashboard)	< 1s pour 200 000 documents
PERF-05	Taille du modele sauvegarde (pipeline complet)	< 50 MB
PERF-06	Consommation memoire du dashboard	< 2 GB RAM

4.2 Fiabilite et disponibilite

ID	Exigence	Critere
FIA-01	Uptime du collecteur	> 95% sur 30 jours
FIA-02	Uptime de MongoDB	> 99% sur 30 jours
FIA-03	Uptime du dashboard	> 95% sur 30 jours
FIA-04	Recovery time apres panne	< 5 minutes (redemarrage automatique Docker)
FIA-05	Aucune perte de donnees en cas de redemarrage	Volumes Docker persistants

4.3 Securite

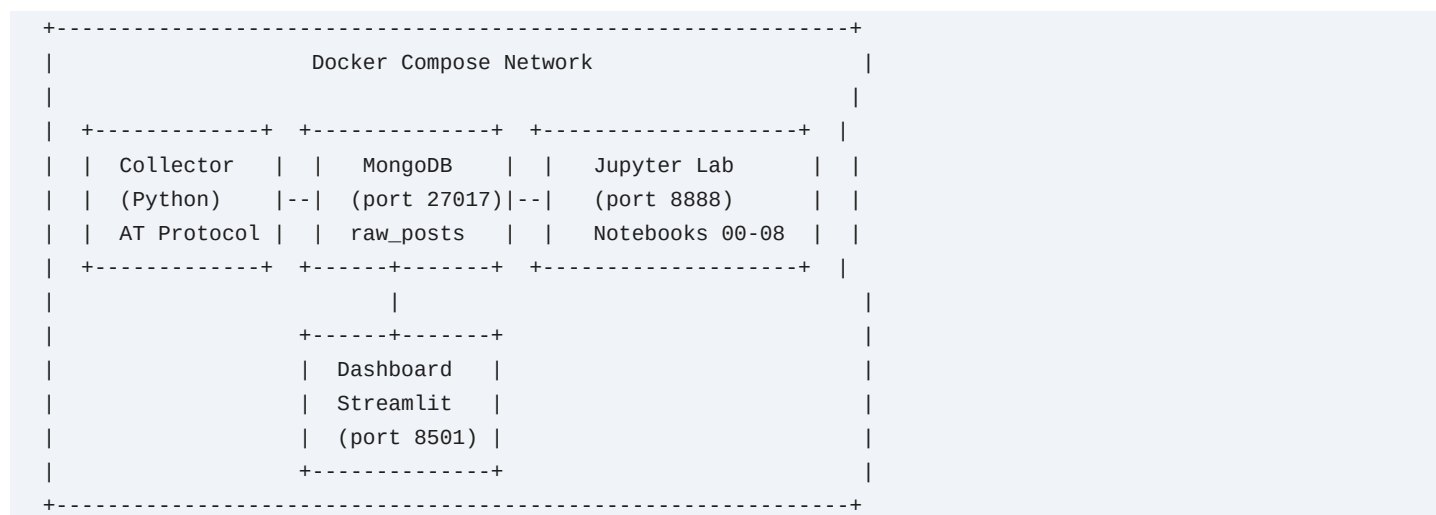
ID	Exigence	Critere
SEC-01	Les identifiants Bluesky doivent etre stockes dans un fichier .env non versionne	.env dans .gitignore
SEC-02	Les identifiants MongoDB ne doivent pas etre en clair dans le code	Variables d'environnement
SEC-03	Le dashboard ne doit pas exposer de donnees sensibles (tokens, mots de passe)	Audit de securite du code
SEC-04	Les images Docker doivent etre scannees pour les vulnerabilites connues	Scan Trivy ou equivalent

4.4 Maintainabilite

ID	Exigence	Critere
MAI-01	Le code doit etre structure en modules separes (collecte, pipeline, dashboard)	Architecture actuelle respectee
MAI-02	Les modeles doivent etre versiones et chargeables par suffixe	load(suffix='expert_v2') fonctionnel
MAI-03	Les notebooks doivent etre numerotes et documentes	00 a 08, cellules markdown presentes
MAI-04	Un guide utilisateur doit etre disponible	docs/guide_utilisateur.md
MAI-05	L'empreinte carbone doit etre mesuree a chaque entrainement	CodeCarbon integre

5. Architecture technique cible

5.1 Vue d'ensemble



5.2 Stack technologique

Couche	Technologie	Version	Justification
Langage	Python	3.9	Compatibilite des librairies ML
Collecte	atproto	derniere	SDK officiel AT Protocol
Stockage	MongoDB	derniere (Docker)	NoSQL adapte aux documents JSON
ML classique	scikit-learn	>= 1.0	Pipeline TF-IDF + LogReg
Deep Learning	PyTorch	>= 2.0	MLP emotions, compatibilite Apple Silicon
NLP	langdetect, transformers	dernieres	Detection de langue, tokenization

Visualisation	Streamlit + Plotly	dernieres	Dashboard interactif
Conteneurisation	Docker + Docker Compose	dernieres	Isolation des services
Monitoring CO2	CodeCarbon	derniere	Mesure empreinte carbone

5.3 Flux de donnees

1. INGESTION

Bluesky API -> collect_bluesky.py -> MongoDB (raw_posts)

Frequence : toutes les 5 minutes

Volume : ~1 000-5 000 posts/jour

2. ENRICHISSEMENT

MongoDB (raw_posts) -> expert_detector.py -> MongoDB (enriched)

Champs ajoutes : ai_score_credibility, prediction_label, ai_emotion, ai_language

3. RESTITUTION

MongoDB (enriched) -> dashboard/app.py -> Navigateur web

Rafraichissement : a chaque chargement de page

5.4 Modeles et artefacts

Artefact	Fichier	Taille	Description
Modele V5	model_expert_v5.pkl	~15 MB	LogisticRegression calibre (15 features ling.)
Vectorizer V5	tfidf_expert_v5.pkl	~30 MB	TF-IDF 30K features bilingue
Metriques V5	metrics_expert_v5.pkl	< 1 MB	Scores CV, matrices de confusion
Modele V4	model_expert_v4.pkl	~15 MB	LogisticRegression calibre (15 features ling.) - archive
Vectorizer V4	tfidf_expert_v4.pkl	~30 MB	TF-IDF 30K features bilingue - archive
Metriques V4	metrics_expert_v4.pkl	< 1 MB	Scores CV, matrices de confusion - archive
CamemBERT FR	camembert_fr.pt	~450 MB	Fine-tuned CamemBERT pour FR court
Emotions	emotion_bilingual.pt	~7 MB	MLP PyTorch 7 classes
Vocabulaire	emotion_vocab_bilingual.pickle	~2 MB	Mapping mot -> token
Encodeur	emotion_label_encoder_bilingual.pickle	< 1 KB	7 labels d'emoions

6. Interfaces entre modules

6.1 Interface Collecteur -> MongoDB

```
{
  "uri": "at://did:plc:xxx/app.bsky.feed.post/yyy",
  "cid": "bafyreid...",
  "text": "Contenu du post",
  "created_at": "2026-03-15T10:30:00Z",
  "search_term": "vaccin",
  "search_lang": "fr",
  "collected_at": "2026-03-15T10:30:05Z",
  "author_did": "did:plc:xxx",
  "author_handle": "user.bsky.social",
  "reply_count": 0,
  "repost_count": 3,
  "like_count": 12,
  "has_image": false,
  "image_url": null,
  "ai_processed": false
}
```

6.2 Interface Pipeline -> MongoDB (enrichissement)

```
{
  "ai_score_credibility": 0.72,
  "prediction_label": "FIABLE",
  "ai_emotion": "neutre",
  "ai_emotion_probabs": {
    "colere": 0.05, "degout": 0.02, "joie": 0.10,
    "neutre": 0.65, "peur": 0.08, "surprise": 0.05, "tristesse": 0.05
  },
  "ai_language": "fr",
  "ai_analysis_log": "v2 pipeline, threshold=0.44",
  "ai_processed": true
}
```

6.3 Interface Pipeline -> Dashboard

Le dashboard charge le pipeline via :

```
detector = ExpertFakeNewsDetector(model_dir='../models', threshold=0.44)
detector.load(suffix='expert_v2')
results = detector.predict(pd.Series(texts))
```

Retour : DataFrame avec colonnes text, language, prediction_label, ai_score_credibility, ai_analysis_log

7. Contraintes et hypotheses

7.1 Contraintes

Type	Contrainte	Impact
Reglementaire	Conformite RGPD obligatoire	Pas de profilage, droit a l'effacement

Reglementaire	Anticipation AI Act (classification risque limite)	Transparence, documentation
Technique	API Bluesky susceptible de changer	Veille et adaptation du collecteur
Technique	Pas de GPU disponible en production	Modeles legers uniquement (pas de Transformers lourds)
Budgetaire	Infrastructure sur machine locale (pas de cloud)	Docker Compose, pas de Kubernetes
Ethique	Le systeme ne doit pas servir a la censure	Labels indicatifs, pas decisionnels

7.2 Hypotheses

Hypothese	Risque si invalidee
Les posts Bluesky sont publics et collectables legalement	Arret de la collecte
Le volume de posts reste < 10 000/jour	Redimensionnement de l'infrastructure
La distribution des fake news reste stable dans le temps	Retraining necessaire
Les utilisateurs du dashboard comprennent les limites du modele	Mauvaise interpretation des scores

8. Criteres d'acceptation globaux

Le projet est considere comme recette lorsque :

- Collecte : Le collecteur fonctionne en continu depuis > 7 jours sans intervention manuelle
- Stockage : La base MongoDB contient > 100 000 posts avec tous les champs obligatoires
- Detection : Le pipeline V5 atteint $F1 \geq 0.90$ en cross-validation ($F1_{global}=0.913$, $F1_{FR}=0.944$, $F1_{FR\ court}=0.904$, $F1_{EN}=0.894$), et < 40% de posts suspects sur Bluesky
- Emotions : Le modele emotionnel produit 7 probabilites coherentes pour chaque texte
- Dashboard : Les 3 pages sont fonctionnelles, l'analyse temps reel repond en < 3s
- Infrastructure : docker-compose up lance les 4 services sans erreur
- Documentation : Guide utilisateur, rapport technique et cahier des charges disponibles
- Green IT : Empreinte carbone mesuree et documentee pour chaque entraînement

9. Livrables attendus

#	Livable	Format	Responsable
L1	Code source complet	Repository Git	Data Engineer + ML Engineer

L2	Modeles entraines (V5 + emotions + CamemBERT)	Fichiers .pkl et .pt	ML Engineer
L3	Dashboard fonctionnel	Application Streamlit	Dashboard Developer
L4	Infrastructure Docker	docker-compose.yml + Dockerfile	DevOps
L5	Notebooks documentes (00 a 08)	Jupyter .ipynb	Data Scientist
L6	Guide utilisateur	Markdown	Chef de Projet
L7	Rapport technique	Markdown	Data Scientist + ML Engineer
L8	Cahier des charges techniques	Markdown (ce document)	Chef de Projet
L9	Document de conformite RGPD	Markdown	Chef de Projet + DPO
L10	Bilan carbone	CSV + rapport	Expert Green IT

10. Glossaire

Terme	Definition
AT Protocol	Protocole decentralise de Bluesky pour l'echange de donnees sociales
TF-IDF	Term Frequency-Inverse Document Frequency, methode de vectorisation de texte
LogReg	Regression Logistique, algorithme de classification lineaire
MLP	Multi-Layer Perceptron, reseau de neurones a couches denses
F1-score	Moyenne harmonique de la precision et du rappel
Cross-validation	Methode d'evaluation par decoupage du dataset en K folds
Domain shift	Ecart de performance entre les donnees d'entrainement et de production
Backoff exponentiel	Strategie de retry avec delai croissant
CodeCarbon	Librairie Python de mesure d'empreinte carbone des calculs
Glassmorphism	Style visuel avec effets de transparence et de flou

Document valide par la Direction Projet - Avril 2026

Reference : CDC-THUM-2026-001 - Version 2.0